WEEK - 12

Multi Threaded Programming and Lambda Expressions

Name: Chandan
B.Tech CSE

Lab 1

1. Question

Write a Java program to create multiple threads and demonstrate different thread priorities using MIN_PRIORITY, NORM_PRIORITY, and MAX_PRIORITY.

2. Program

```
class MyThread extends Thread {
    MyThread(String name) {
        super(name);
    }

    public void run() {
        System.out.println(getName() + " running with priority " + getPriority());
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread-Min");
        MyThread t2 = new MyThread("Thread-Norm");
        MyThread t3 = new MyThread("Thread-Max");

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
        t3.start();
    }
}
```

3. Output

```
Thread-Norm running with priority 5
Thread-Min running with priority 1
Thread-Max running with priority 10
```

Lab 2

1. Question

Write a Java program to create three threads with different priorities and observe their execution order.

2. Program

```
class Task extends Thread {
    Task(String name, int priority) {
        super(name);
        setPriority(priority);
    }

    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " (priority " + getPriority() + ") - step " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Task t1 = new Task("Low", 2);
        Task t2 = new Task("Medium", 5);
        Task t3 = new Task("High", 9);

        t1.start();
        t2.start();
        t3.start();
    }
}
```

3. Output

```
Low (priority 2) - step 1
Low (priority 2) - step 2
Low (priority 2) - step 3
Medium (priority 5) - step 1
Medium (priority 5) - step 2
Medium (priority 5) - step 3
High (priority 9) - step 1
High (priority 9) - step 2
High (priority 9) - step 3
```

Lab 3

1. Question

Write a Java program to demonstrate synchronization using a synchronized method for accessing a shared counter.

2. Program

```
class Counter {
    int count = 0;

    synchronized void increment() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) c.increment();
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) c.increment();
        });

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final count = " + c.count);
    }
}
```

3. Output

```
Final count = 2000
```

Lab 4

1. Question

Write a Java program to demonstrate synchronization using a synchronized block.

2. Program

```
class Printer {
    void print(String name) {
        synchronized (this) {
            for (int i = 1; i <= 3; i++) {
                System.out.println(name + " is printing page " + i);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Printer p = new Printer();

        Thread t1 = new Thread(() -> p.print("User1"));
        Thread t2 = new Thread(() -> p.print("User2"));

        t1.start();
        t2.start();
    }
}
```

3. Output

```
User1 is printing page 1
User1 is printing page 2
User1 is printing page 3
User2 is printing page 1
User2 is printing page 2
User2 is printing page 3
```

Lab 5

1. Question

Write a Java program to demonstrate a race condition and eliminate it using synchronization.

2. Program

```
class Counter {
    int count = 0;

    void unsafeIncrement() {
        count++;
    }

    synchronized void safeIncrement() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter c1 = new Counter();
        Thread a = new Thread(() -> { for (int i = 0; i < 1000000; i++) c1.unsafeIncrement(); });
        Thread b = new Thread(() -> { for (int i = 0; i < 1000000; i++) c1.unsafeIncrement(); });
        a.start(); b.start();
        a.join(); b.join();
        System.out.println("Without synchronization: " + c1.count);

        Counter c2 = new Counter();
        Thread x = new Thread(() -> { for (int i = 0; i < 1000000; i++) c2.safeIncrement(); });
        Thread y = new Thread(() -> { for (int i = 0; i < 1000000; i++) c2.safeIncrement(); });
        x.start(); y.start();
        x.join(); y.join();
        System.out.println("With synchronization   : " + c2.count);
    }
}
```

3. Output

```
Without synchronization: 1859923
With synchronization   : 2000000
```

Lab 6

1. Question

Write a Java program to implement a synchronized ticket-booking system where multiple threads attempt to book tickets simultaneously.

2. Program

```
class TicketCounter {
   int tickets = 5;

   synchronized void bookTicket(String name) {
      if (tickets > 0) {
         System.out.println(name + " booked ticket no. " + tickets);
         tickets--;
      } else {
         System.out.println(name + ": Sorry, no tickets available");
      }
   }
}

public class Main {
   public static void main(String[] args) {
      TicketCounter tc = new TicketCounter();

      for (int i = 1; i <= 7; i++) {
         String name = "Person" + i;
         new Thread(() -> tc.bookTicket(name)).start();
      }
   }
}
```

3. Output

```
Person1 booked ticket no. 5
Person6 booked ticket no. 4
Person5 booked ticket no. 3
Person4 booked ticket no. 2
Person7 booked ticket no. 1
Person2: Sorry, no tickets available
Person3: Sorry, no tickets available
```

Lab 7

1. Question

Write a Java program to introduce lambda expressions by implementing a functional interface for addition, subtraction, multiplication, and division.

2. Program

```
interface Calculator {
    double operate(double a, double b);
}

public class Main {
    public static void main(String[] args) {
        Calculator add = (a, b) -> a + b;
        Calculator sub = (a, b) -> a - b;
        Calculator mul = (a, b) -> a * b;
        Calculator div = (a, b) -> a / b;

        System.out.println("Addition       : " + add.operate(20, 5));
        System.out.println("Subtraction    : " + sub.operate(20, 5));
        System.out.println("Multiplication : " + mul.operate(20, 5));
        System.out.println("Division       : " + div.operate(20, 5));
    }
}
```

3. Output

```
Addition       : 25.0
Subtraction    : 15.0
Multiplication : 100.0
Division       : 4.0
```

Lab 8

1. Question

Write a Java program to demonstrate a lambda expression with a single parameter and multiple parameters.

2. Program

```
interface Square {
   int calculate(int n);
}

interface Sum {
   int add(int a, int b);
}

public class Main {
   public static void main(String[] args) {
      Square s = n -> n * n;
      Sum sum = (a, b) -> a + b;

      System.out.println("Square of 6 = " + s.calculate(6));
      System.out.println("Sum of 10 and 15 = " + sum.add(10, 15));
   }
}
```

3. Output

```
Square of 6 = 36
Sum of 10 and 15 = 25
```

Lab 9

1. Question

Write a Java program to demonstrate a block lambda expression containing multiple statements.

2. Program

```
interface Greeting {
   void show(String name);
}

public class Main {
   public static void main(String[] args) {
      Greeting g = (name) -> {
         System.out.println("Hello " + name);
         System.out.println("Welcome to Java Lambda Expressions");
         System.out.println("Have a nice day!");
      };

      g.show("Chandan");
   }
}
```

3. Output

```
Hello Chandan
Welcome to Java Lambda Expressions
Have a nice day!
```

Lab 10

1. Question

Write a Java program using a block lambda expression to find the largest of two numbers and return the result.

2. Program

```
interface Largest {
   int find(int a, int b);
}

public class Main {
   public static void main(String[] args) {
      Largest l = (a, b) -> {
         if (a > b) {
            return a;
         } else {
            return b;
         }
      };

      System.out.println("Largest of 25 and 40 = " + l.find(25, 40));
   }
}
```

3. Output

```
Largest of 25 and 40 = 40
```

Lab 11

1. Question

Write a Java program to create a generic functional interface and implement it using lambda expressions for different data types.

2. Program

```
interface Show<T> {
    T display(T value);
}

public class Main {
    public static void main(String[] args) {
        Show<Integer> i = (n) -> n * 2;
        Show<String> s = (str) -> str.toUpperCase();
        Show<Double> d = (x) -> x + 0.5;

        System.out.println("Integer : " + i.display(10));
        System.out.println("String  : " + s.display("java"));
        System.out.println("Double  : " + d.display(4.5));
    }
}
```

3. Output

```
Integer : 20
String  : JAVA
Double  : 5.0
```

Lab 12

1. Question

Write a Java program to create a generic functional interface that accepts two values and performs an operation using a lambda expression.

2. Program

```
interface Operation<T> {
    T apply(T a, T b);
}

public class Main {
    public static void main(String[] args) {
        Operation<Integer> add = (a, b) -> a + b;
        Operation<String> join = (a, b) -> a + " " + b;
        Operation<Double> mul = (a, b) -> a * b;

        System.out.println("Integer addition : " + add.apply(10, 20));
        System.out.println("String joining   : " + join.apply("Hello", "World"));
        System.out.println("Double multiply  : " + mul.apply(2.5, 4.0));
    }
}
```

3. Output

```
Integer addition : 30
String joining   : Hello World
Double multiply  : 10.0
```

Lab 13

1. Question

Write a Java program to use lambda expressions to display and filter elements of an ArrayList.

2. Program

```
import java.util.ArrayList;

public class Main {
   public static void main(String[] args) {
      ArrayList<Integer> list = new ArrayList<>();
      list.add(5);
      list.add(12);
      list.add(8);
      list.add(21);
      list.add(16);

      System.out.println("All elements:");
      list.forEach(n -> System.out.println(n));

      System.out.println("Even numbers:");
      list.stream().filter(n -> n % 2 == 0).forEach(n -> System.out.println(n));
   }
}
```

3. Output

```
All elements:
5
12
8
21
16
Even numbers:
12
8
16
```

Lab 14

1. Question

Write a Java program to sort a list of integers and strings using lambda expressions.

2. Program

```
import java.util.ArrayList;

public class Main {
   public static void main(String[] args) {
      ArrayList<Integer> nums = new ArrayList<>();
      nums.add(30);
      nums.add(10);
      nums.add(50);
      nums.add(20);

      ArrayList<String> names = new ArrayList<>();
      names.add("Ravi");
      names.add("Anil");
      names.add("Chandan");
      names.add("Bhanu");

      nums.sort((a, b) -> a - b);
      names.sort((a, b) -> a.compareTo(b));

      System.out.println("Sorted integers: " + nums);
      System.out.println("Sorted strings : " + names);
   }
}
```

3. Output

```
Sorted integers: [10, 20, 30, 50]
Sorted strings : [Anil, Bhanu, Chandan, Ravi]
```

Lab 15

1. Question

Write a Java program to create and execute a thread using a lambda expression.

2. Program

```
public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Lambda thread running: " + i);
            }
        });

        t.start();
    }
}
```

3. Output

```
Lambda thread running: 1
Lambda thread running: 2
Lambda thread running: 3
Lambda thread running: 4
Lambda thread running: 5
```